Lahore University of Management Sciences

Syed Babar Ali School of Science and Engineering

CS 487 / CS 5603 — Cloud Development · Fall 2026

Module 2 — Client–Server Software, Web & FTP Servers

A self-contained tutorial. No prior systems background assumed.

Prepared by Muhammad Kashif and M. Usman Tahir

1. Scope and Prerequisites

Every website, every app that shows fresh data, every "Sign in with Microsoft" button rests on one idea: **one computer asks another for something and gets an answer back.** This module is the detail — what the question looks like, what the answer looks like, which door it arrives at, how the conversation is kept private, and how all of it maps onto Azure.

By the end you should be able to explain what happens between pressing Enter on a URL and seeing a page; read an HTTP exchange and say what went wrong; explain what a port is and why 443 keeps appearing; describe stateless vs stateful and why cloud platforms depend on the distinction; read an Nginx, Apache, or IIS config without panicking; diagnose a `502` from a log file; explain what a TLS certificate does and does not prove; and deploy a real application to Azure App Service.

Tool	Why	Where
Browser (Chrome/Edge)	Its built-in **DevTools** is the single most useful debugging tool here	Already installed
`curl`	Send HTTP requests by hand	Built into Windows 10+, macOS, Linux
Azure CLI (`az`)	Drive Azure from a terminal	learn.microsoft.com/cli/azure
Azure account	Actually deploy things	Azure for Students — free credit, no card
VS Code	Editor	code.visualstudio.com
Docker Desktop *(optional)*	Run Nginx locally without installing it	docker.com

Where a command line is shown starting with `$`, the `$` is the prompt, not part of the command. "Terminal", "shell", and "command line" all mean the same window — PowerShell or Windows Terminal on Windows, Terminal on macOS and Linux.

2. The Client–Server Model

2.1 The idea

Think of a restaurant. **You** are the client: you want food and neither know nor need to know how it is made. **The kitchen** is the server: it holds the ingredients, equipment, and recipes. **The waiter** carries a request in and a response out — a plate, or "sorry, we're out of chicken".

Three properties carry over exactly to computers:

1. **You never enter the kitchen.** You do not know how many cooks are inside. This is *abstraction*, and it is why the cloud works at all — a provider can replace every machine overnight and you would never know.

2. **One kitchen serves many tables.** A server is not a special magic machine; it is a computer whose job is answering many requests.
3. **You always speak first.** The kitchen does not send food you did not order. In the classic model the client always initiates.

2.2 The details

A **client** is any program that initiates a request — your browser, a phone app, `curl`, a Python script. A **server** is any program that listens for requests and returns responses. Note *program*, not machine: one machine can run five servers, and one logical server can span a thousand machines.

```
CLIENT                                          SERVER
  |  1. Open a connection  ---------------------> |
  |  2. "GET /products/42 please"  -------------> |
  |                                               | 3. Is this user allowed?
  |                                               |    Fetch product 42, format answer
  |  <---------------- 4. "200 OK, here is the data"
  |  5. Connection closed or reused               |
```

> **THE KEY PROPERTY**
> The server is **passive**. It sits in a loop doing nothing, waiting. A server not receiving requests is burning electricity for nothing — precisely the observation that produced serverless computing (§6.5).

2.3 What happens when you type a URL

You type `https://www.lums.edu.pk/courses` and press Enter.

```
https://www.lums.edu.pk:443/courses?page=2#top
  scheme        host       port   path    query  fragment
```

Piece	Meaning
scheme	Which protocol to speak. `https` = HTTP with encryption.
host	*Which computer*, as a human-friendly name.
port	*Which door* on that computer. Usually hidden: `http` defaults to 80, `https` to 443.
path	*Which resource*. Looks like a file path; often is not a real file.
query	Extra `key=value` parameters joined by `&`.
fragment	Position within the page. **Never sent to the server** — the browser keeps it.

1. **Parse the URL** into the pieces above.
2. **DNS lookup.** Computers route by number, not name, so `www.lums.edu.pk` must become an IP like `104.18.32.7`. DNS is the phone book of the internet. Try `nslookup www.lums.edu.pk`.
3. **Open a TCP connection** to that IP on port 443. TCP guarantees the bytes arrive, in order, uncorrupted, via a three-step handshake (`SYN` → `SYN-ACK` → `ACK`): "can you hear me?" / "yes, can you hear me?" / "yes".
4. **TLS handshake** — because we used `https`, encryption is negotiated *before* any real data moves (§9).
5. **Send the HTTP request** — plain text, readable by eye.
6. **The server works** — reads a file, or queries a database and renders HTML, or forwards to another server entirely (§7.5).
7. **Receive the response.**
8. **Render, then repeat.** The browser finds references to CSS, JS, images, fonts, and *fires a new request for each*. One "page load" is typically 30–120 request/response cycles.

> **TRY IT**
> Open any site. Press **F12** → **Network** → reload. You are now looking at every request the browser made. Click one to see Headers, Response, and Timing. Get comfortable here now — you will live in this panel.

2.4 Why cloud people care

The client–server split is *why the cloud exists*. Because the client never sees inside the kitchen, the provider is free to run your server on any machine in any datacentre, run twenty copies behind one address, kill a crashed instance and start a fresh one mid-traffic, and move you during maintenance. None of that is possible if clients depend on a specific machine. This is the reason for §6.2.

Two variations worth recognising: **peer-to-peer**, where every node is both client and server (BitTorrent, blockchain); and **server-initiated messages**, which plain HTTP cannot do — hence **WebSockets** (persistent two-way, used by chat and live dashboards), **Server-Sent Events** (one-way server→client, used by AI interfaces streaming tokens), and **polling** (client asks repeatedly; simple, wasteful).

3. HTTP and HTTPS

A **protocol** is an agreed set of rules for a conversation. Call a service line and there is an unspoken protocol: greeting, your problem, your account number. Recite the account number before they pick up and the conversation fails — not because the information was wrong, but because the order and format were.

HTTP is that agreed format for the web, and it is a text format, which means you can read it with your eyes. **HTTPS** is the identical protocol wrapped in an encrypted envelope. There is no separate "HTTPS protocol"; it is HTTP over TLS.

3.1 Anatomy

Every request has the same four parts, and every response mirrors it with an outcome instead of an intention:

```
POST /api/v1/students HTTP/1.1              <- 1. START LINE (method, path, version)
Host: api.university.edu                    <- 2. HEADERS (metadata)
Content-Type: application/json
Authorization: Bearer eyJhbGciOi...
                                            <- 3. BLANK LINE (mandatory)
{"name": "Ayesha Khan", "rollNo": "27100431"}   <- 4. BODY (optional)
```

```
HTTP/1.1 201 Created                        <- STATUS LINE
Content-Type: application/json
Location: /api/v1/students/8891

{"id": 8891, "name": "Ayesha Khan"}
```

The blank line is absolutely required — a missing blank line is a malformed request. `GET` and `DELETE` normally have no body; `POST`, `PUT`, and `PATCH` usually do.

3.2 Methods

Method	Means	Body?	Safe?	Idempotent?
`GET`	Give me this. Change nothing.	No	Yes	Yes
`POST`	Here is data; create or do something.	Yes	No	No
`PUT`	Replace this entirely with what I send.	Yes	No	Yes
`PATCH`	Change *part* of this.	Yes	No	Usually
`DELETE`	Remove this.	No	No	Yes
`HEAD`	Like GET, headers only.	No	Yes	Yes
`OPTIONS`	What methods does this endpoint support?	No	Yes	Yes

Safe means calling it changes nothing on the server. This is why browsers pre-fetch `GET` URLs and crawlers only issue `GET`. **Never** build `/deleteUser?id=5` responding to `GET` — a crawler will eventually delete your user table. This has happened to real companies.

Idempotent means calling it once or five times leaves the same final state. `DELETE /students/8891` five times: gone after the first, the rest do nothing. `POST /students` five times: five students. Why it matters practically — *the network is unreliable*. If a request times out you do not know whether the server processed it. Idempotent operations can simply be retried; non-idempotent ones might double-charge a card. Cloud systems retry constantly, so idempotency is a design requirement, not trivia.

Where the data goes. `GET` puts it in the URL — visible in browser history, server logs, and proxy logs (**never put passwords or tokens in a URL**), capped near 2,000 characters, but bookmarkable and shareable. `POST` puts it in the body — not logged by default, no length limit, not bookmarkable. Note that `POST` is **not encrypted**; it is merely not visible in the URL bar. Only HTTPS encrypts. A password sent by `POST` over plain HTTP is fully readable by anyone on the network.

3.3 Status codes

The first digit gives the category, and the **4xx/5xx split is the single most useful debugging distinction in this module: 4xx means fix your request, 5xx means fix the server.**

Code	Name	When you see it
2xx — Success		
`200`	OK	Standard success; body has what you asked for.
`201`	Created	Your `POST` created something. Check the `Location` header.
`202`	Accepted	"Got it, we'll process later." Common in queue-based systems (§6.4).
`204`	No Content	Success, nothing to send back. Typical for `DELETE`.
3xx — Redirection		
`301`	Moved Permanently	Dead forever, use the new URL. Cached **aggressively** — a wrong 301 can strand users for months.
`302`	Found (temporary)	Go here for now, keep asking me.
`304`	Not Modified	"Your cached copy is still good." Huge bandwidth saver.
4xx — You made the mistake		
`400`	Bad Request	Malformed JSON, missing field, wrong type.
`401`	Unauthorized	**Badly named — it means *unauthenticated*.** Missing or expired token.
`403`	Forbidden	We know who you are; you are not allowed.
`404`	Not Found	Wrong path, or the resource genuinely does not exist.
`405`	Method Not Allowed	Right URL, wrong verb.
`409`	Conflict	Duplicate. "That email is already registered."
`413`	Payload Too Large	Upload exceeded the limit. Very common behind Nginx.
`415`	Unsupported Media Type	You sent XML; it wanted JSON. Usually a missing `Content-Type`.
`429`	Too Many Requests	Rate limited. Check the `Retry-After` header.
5xx — The server made the mistake		
`500`	Internal Server Error	Unhandled exception in the application code.
`502`	Bad Gateway	A proxy reached for the app behind it and got nothing or garbage. **See §8.4.**
`503`	Service Unavailable	Up but overloaded, or deliberately in maintenance.
`504`	Gateway Timeout	Proxy reached the app, app took too long.

TWO DISTINCTIONS WORTH MEMORISING

401 vs 403: 401 = "Who are you?" · 403 = "I know who you are, and no."

502 vs 504: 502 = "the thing behind me is *broken*" · 504 = "the thing behind me is *slow*".

3.4 Headers

Dir.	Header	Purpose
Req	`Host`	Which website you want. **Mandatory in HTTP/1.1.** One IP can host 500 sites; this is how the server knows which. Foundation of virtual hosting (§7.4).
Req	`Accept` / `Accept-Encoding`	Formats and compression the client handles. `gzip, br` can shrink responses 70%+.
Req	`Authorization`	Credentials, usually `Bearer <token>`.
Req	`Content-Type`	Format of the body *you are sending*.
Req	`User-Agent` / `Cookie` / `Referer`	Client software; stored values sent back; the page you came from. (`Referer` has been misspelled in the spec since 1996. Nobody is fixing it.)
Resp	`Content-Type`	Format of the body. **Wrong value and browsers do strange things** — e.g. downloading your JSON as a file.
Resp	`Location`	Where to go next (with 3xx and 201).
Resp	`Cache-Control` / `ETag`	How long this may be cached; a content fingerprint enabling `304`.
Resp	`Set-Cookie`	"Store this and send it back next time."
Resp	`Access-Control-Allow-Origin`	CORS — which other sites' JavaScript may read this response.
Resp	`Strict-Transport-Security`	"Only ever talk to me over HTTPS." (§9.5)
Resp	`X-Powered-By` / `Server`	Reveals your stack. **Turn off in production** — free reconnaissance for attackers.

3.5 CORS, because it will bite you

By default, JavaScript on `https://mysite.com` is **not allowed to read** a response from `https://api.othersite.com`. This protects you: otherwise a malicious page could quietly call your bank's API using your logged-in session and read the results. To permit it, the *server* must send `Access-Control-Allow-Origin: https://mysite.com`.

> **THREE THINGS STUDENTS GET WRONG EVERY SEMESTER**
>
> 1. **CORS is enforced by the browser, not the server.** `curl` will happily fetch the data; your JavaScript will not. *If it works in curl or Postman but fails in the browser, suspect CORS.*
> 2. **The fix belongs on the API server**, not in frontend code. You cannot "turn off CORS" from the client.
> 3. For non-simple requests the browser sends a **preflight** `OPTIONS` first. Mysterious `OPTIONS` requests in your logs are this.

```
az webapp cors add --resource-group myRG --name myApp --allowed-origins https://mysite.com
```

Versions, briefly: HTTP/1.1 (1997) added reusable connections and the `Host` header; HTTP/2 (2015) made it binary and multiplexed; HTTP/3 (2022) runs over QUIC/UDP and handles flaky mobile networks far better. You rarely choose — browsers and Azure negotiate automatically. The *semantics* (methods, codes, headers) are identical across all versions; only the wire encoding changed.

TRY IT

```
curl -I https://www.microsoft.com                  # response headers only
curl -v https://api.github.com/users/octocat       # the full conversation
curl -i https://api.github.com/this-does-not-exist # force a 404
curl -X POST https://httpbin.org/post \
  -H "Content-Type: application/json" -d '{"course": "CS 487"}'
```

`httpbin.org` echoes your request back — the best HTTP sandbox there is. Try `httpbin.org/status/502` to summon any status code on demand.

4. REST APIs and JSON

An **API** is a website for programs instead of people. When you visit a weather site you get icons and ads; when a program wants the same weather it wants the number `34` and the word `Sunny`. **REST** is a set of conventions for designing those APIs — not a technology you install, an agreement about style.

4.1 Nouns and verbs

THE SINGLE INSIGHT OF REST

The URL is a noun. The HTTP method is the verb.

```
BAD (verbs in the URL)              GOOD (one noun, six verbs)
GET  /getAllStudents                GET    /students          list
POST /createNewStudent              POST   /students          create
POST /updateStudentName?id=42       GET    /students/42       read one
GET  /deleteStudent?id=42  <- unsafe PUT   /students/42       replace
                                    PATCH  /students/42       partial update
                                    DELETE /students/42       delete
```

The second list is smaller to learn: once you know the noun you already know all six operations. That predictability is the whole point. **Nesting** expresses relationships (`GET /students/42/enrollments`, `DELETE /students/42/enrollments/CS487`). **Query parameters** filter, sort, and paginate — they never identify a resource (`GET /students?year=4&sort=name&page=2`).

4.2 JSON

Despite the name, JSON has nothing to do with JavaScript any more — every language reads and writes it. There are exactly six types:

Type	Looks like	Rules
Object	`{ }`	Unordered `"key": value` pairs. Keys **must** be double-quoted.
Array	`[ ]`	Ordered list, comma-separated.
String	`"text"`	**Double quotes only.** Single quotes are invalid JSON.
Number	`42`, `3.72`	No quotes, no leading `+`, no `NaN`, no `Infinity`.
Boolean	`true` / `false`	Lowercase. Not `True`, not `TRUE`.
Null	`null`	Lowercase. "Exists but has no value."

THE FOUR MISTAKES EVERYONE MAKES

```
{
  'name': 'Ayesha',       <- single quotes are not JSON
  "cgpa": 3.72,           <- trailing comma before } is not allowed
  // a comment            <- JSON has no comments. None. Ever.
  "date": 2026-08-27      <- dates are not a JSON type; use a string "2026-08-27"
}
```

If an API returns `400` and you cannot see why, paste the body into *jsonlint.com* before doing anything else. Ninety percent of the time it is a comma.

4.3 A worked example

```
POST /api/v1/students          HTTP/1.1        HTTP/1.1 201 Created
Content-Type: application/json                 Location: /api/v1/students/8891
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
                                               {
{                                                "id": 8891,
  "name": "Ayesha Khan",                         "name": "Ayesha Khan",
  "rollNo": "27100431",                          "rollNo": "27100431",
  "program": "BS Computer Science"               "createdAt": "2026-08-27T09:14:22Z"
}                                              }
```

Three professional touches: the status is `201`, not `200`; the `Location` header says where the new thing lives; and the server filled in `id` and `createdAt`, fields the client never sent.

When returning a list, wrap it: `{"data": [...], "pagination": {...}}`. It looks like extra work, but it lets you add pagination, warnings, or links later without breaking every client. Returning a bare `[...]` array is a decision you will regret.

And an error, done properly — telling the client *what* to fix, plus a `traceId` so support can find the exact log entry:

```
HTTP/1.1 422 Unprocessable Entity
{
  "error": {
    "code": "VALIDATION_FAILED",
    "message": "One or more fields were invalid.",
    "details": [ { "field": "rollNo", "issue": "Must be exactly 8 digits." } ],
    "traceId": "0HMV9K2P4L1"
  }
}
```

Compare with the useless but extremely common alternative: `500` with an empty body.

4.4 Authentication

API key — a long random string in a header (`X-API-Key: sk_live_...`). Identifies the *application*, not a user. If it leaks it is leaked until rotated. Never commit one to Git.

Bearer token / JWT — issued after login, expires. A JWT is three base64 chunks separated by dots. **The payload is readable by anyone** (paste one into jwt.io) — it is *signed*, not *encrypted*. Never put secrets in a JWT payload; the signature is what makes it tamper-proof.

OAuth 2.0 — the "Sign in with Microsoft" flow. The user authenticates with the identity provider, which hands your app a token; your app never sees the password. This is what **Microsoft Entra ID** implements and what you will use for RBAC later in this course.

> **WHY CLOUD PEOPLE CARE**
> Cloud platforms *are* REST APIs. When you click in the Azure Portal, the portal is a JavaScript client calling `management.azure.com`. When you run `az vm create`, the CLI makes the same calls. When Terraform provisions infrastructure, same calls. There is no hidden magic path — which is exactly why everything in Azure is scriptable.

5. Ports and Sockets

Your server has one IP address but may run a website, a database, a mail service, and a file server at once. When a packet arrives, how does the OS know which program it is for?

Think of an office building. The **street address** is the IP — it gets mail to the right building. The **suite number** is the port — it gets mail to the right office. The **doorbell being answered** is a program *listening* on that port. Mail addressed to a suite where nobody works is returned to sender: that is "connection refused".

A **port** is just a number from 0 to 65535 with no physical existence. A **socket** is protocol + IP + port, and a full connection is a pair of sockets:

```
203.0.113.9:51234  <----------------->  20.51.3.44:443
  (your laptop)                          (the server)
  ephemeral port                         listening port
```

5.1 Listening vs ephemeral ports

Listening (server side) — fixed, known in advance, published. The server *binds* to it and waits. It must be predictable because clients need to know where to knock.

Ephemeral (client side) — temporary, random, assigned by your OS for one connection then discarded, typically 49152–65535. You never choose it and never care what it is.

Why do clients need a port at all? Because you have twelve tabs open. When responses flood back to your single IP, the ephemeral port is how your OS knows that *this* response belongs to *that* tab.

Port	Service	Notes
22	SSH / SFTP / SCP	Encrypted remote access **and** file transfer.
53	DNS	Name → IP lookup.
80	**HTTP**	Unencrypted web.
443	**HTTPS**	Encrypted web. The one you will use constantly.
20, 21 / 990	FTP / FTPS	See §10.
1433 / 3306 / 5432	SQL Server / MySQL / PostgreSQL	Azure SQL uses 1433.
3389	RDP	Remote Desktop into a Windows VM.
6379 / 27017	Redis / MongoDB	Azure Cache for Redis; Cosmos DB Mongo API.
8080 / 8000	HTTP alternate	Dev servers, app servers behind a proxy.

Ports 0–1023 are "well-known"; on Linux and macOS, binding to them requires administrator privileges — precisely why you run a dev server on 8080 and let a real web server handle 80. **Memorise 80, 443, 22, 3389, 1433.**

5.2 Binding, and the error you will definitely see

Only **one** process can listen on a given IP + port at a time. Hence `EADDRINUSE: address already in use` or `nginx: bind() to 0.0.0.0:80 failed (98: Address already in use)`. Something is already in that apartment; find it and evict it, or move.

```
sudo lsof -i :8080                # Linux/macOS - who owns port 8080?
netstat -ano | findstr :8080      # Windows - then taskkill /PID 12345 /F
```

5.3 `127.0.0.1` vs `0.0.0.0`

Bind address	Means	Reachable from
`127.0.0.1` (localhost)	Loopback only	**This machine only.** Nothing external can connect.
`0.0.0.0`	All interfaces	Anyone who can reach the machine.

> **THE CLASSIC BEGINNER BUG**
>
> *"My app works on the VM when I curl it, but I can't reach it from my laptop."* Almost always: the app bound to `127.0.0.1`. Bind to `0.0.0.0` instead. It is also a legitimate security technique in reverse — bind your database to `127.0.0.1` and it becomes unreachable from the internet, no firewall required.

5.4 Firewalls and Network Security Groups

Even with a program listening correctly, traffic can be blocked at three layers, and a request must pass **every** one:

```
Internet -> [ Azure NSG: allow/deny by port, protocol, source IP ]
         -> [ OS firewall: ufw / iptables / Windows Defender ]
         -> [ Your app, listening on 0.0.0.0:80 ]
```

A brand-new Azure Linux VM allows SSH (22) and nothing else. Install Nginx and cannot reach it? You have not opened port 80 in the NSG.

```
az vm open-port --resource-group myRG --name myVM --port 80 --priority 900
```

6. Application Architecture

6.1 N-tier

A **2-tier** design has a fat client talking straight to a database — normal in the 1990s, now bad practice, because every client holds database credentials and business rules are duplicated across every installation. The standard web architecture is **3-tier**:

```
PRESENTATION   Browser, mobile app, React/Angular SPA
               Show things to humans, collect input
                    |  HTTPS / JSON
APPLICATION    Node.js / .NET / Django / Spring API
               Validate, authorise, apply business rules
               Azure: App Service, Container Apps, AKS, Functions
                    |  SQL / driver protocol
DATA           Azure SQL, Cosmos DB, PostgreSQL, Blob Storage
               Store it, keep it consistent, keep it safe
```

Four reasons to split: **independent scaling** (add API servers without touching the database — in the cloud you pay per resource, so scaling only what is slow is real money); **security depth** (the database sits on a private network with no public IP, reachable only by the app tier); **independent replacement** (rewrite the frontend; the

API does not change); and **team boundaries** (frontend, backend, and DBA work in parallel against agreed contracts).

> **MONOLITH VS MICROSERVICES**
> Microservices buy independent deployment and scaling at the cost of enormous operational complexity: distributed debugging, network failures between your own components, data consistency across services. **Advice for this course, and honestly for the first years of your career:** start with a well-structured monolith and split only when a specific piece has a genuinely different scaling or release profile. "We use microservices" is not an architecture; it is a bill.

6.2 Stateless vs stateful — the most important idea here

State is information the server remembers *between requests*. A stateful service keeps it in its own memory, and the moment you run more than one copy, it breaks:

```
STATEFUL                                    STATELESS
Req 1 -> Server A: "Login as Ayesha"        Req 1 -> Server A: returns a signed token
         A remembers session123 = Ayesha
Req 2 -> Server A: "Show my cart"      OK   Req 2 -> Server B: token -> reads cart
Req 3 -> Server B: "Show my cart"    FAILS           from Redis            OK
         "session123? Never heard of it"    Req 3 -> Server C: same            OK
```

Capability	Stateless	Stateful
Add instances during a traffic spike	Trivial	Painful
Remove instances when quiet (save money)	Trivial	Loses user sessions
Kill and replace a crashed instance	Invisible to users	Users get logged out
Deploy a new version with zero downtime	Standard	Requires careful draining
Distribute across Azure regions	Straightforward	Hard

> **THE SINGLE MOST USEFUL RULE OF THUMB IN CLOUD DEVELOPMENT**
> *Treat every server as if it will be destroyed without warning in the next five minutes.* If that would lose data, the data is in the wrong place. This is "cattle, not pets" — you do not name a server and nurse it back to health; you shoot it and start another.

State	Put it in
User session	Azure Cache for Redis, or a signed cookie / JWT
Uploaded files	Azure Blob Storage — **never** the local disk of an app server
Application data	Azure SQL / Cosmos DB
Background jobs	Azure Service Bus / Storage Queues
Configuration & secrets	App Settings / Azure Key Vault

Sticky sessions (session affinity) are the load balancer's escape hatch: pin each user to one instance so stateful apps keep working. Azure App Service calls this *ARR Affinity* and enables it by default. It works, but it undermines even load distribution and still loses sessions on restart. If your app is properly stateless, turn it off:

```
az webapp update --resource-group myRG --name myApp --client-affinity-enabled false
```

6.3 Load balancing

A load balancer sits in front of several identical instances and distributes requests — **round robin** (take turns), **least connections** (send to the least busy), or **IP hash** (same client → same server). It also runs **health checks**, pinging each instance every few seconds (typically `GET /health`) and silently removing any that fails. This is how a crashed server becomes a non-event instead of an outage.

Azure service	Layer	Use for
Azure Load Balancer	4 (TCP/UDP)	Raw traffic to VMs. Fast, protocol-agnostic.
Application Gateway	7 (HTTP), regional	Path-based routing, TLS termination, Web Application Firewall.
Azure Front Door	7, global	Routing users to the nearest region, global CDN, DDoS protection.
built-in	—	App Service and Container Apps load-balance across instances automatically.

Layer 4 means it only sees IP and port. *Layer 7* means it can read the HTTP request — path, headers, hostname — and route accordingly (`/api/*` → API pool, everything else → web pool).

6.4 Message queues

The problem. A user uploads a 200 MB video and your API must transcode it — four minutes. Inside the HTTP request that means: the browser stares at a spinner, the request times out (most proxies give up after 30–240 seconds, producing `504`), and one worker thread is blocked the whole time. Ten uploads and your API is dead.

The solution. Do not do the work during the request. Write down that the work *needs doing* and answer immediately.

```
                  +-----------------+
Browser -> [ API ] ->  QUEUE         -> [ Worker 1 ] -> transcode -> Blob Storage
   ^               | job1 job2 job3 | -> [ Worker 2 ]
   +-- 202 Accepted, {"jobId": "abc"} -> [ Worker 3 ]
```

The API saves the file, drops a message on the queue, and returns `202 Accepted` with a job ID — total time 200 milliseconds. Separate workers pull jobs at their own pace; the user polls `GET /jobs/abc`. What a queue buys you: **decoupling** (the API neither knows nor cares how many workers exist), **buffering** (a spike fills the queue instead of crashing the system), **reliability** (a worker dying mid-job makes the message visible again for retry), **independent scaling** (queue 10,000 deep? start 50 workers; empty? scale to zero), and **dead-lettering** (messages failing repeatedly move aside for a human instead of poisoning the pipeline).

Azure service	When to use
Storage Queues	Simple, cheap, huge scale. Start here.
Service Bus	Enterprise messaging: topics & subscriptions, sessions, ordering, transactions, dead-letter queues.
Event Hubs	Massive telemetry streams (millions/sec). IoT and log ingestion, not task queues.
Event Grid	Reacting to events ("a blob was uploaded → run this function").

> **DESIGN NOTE**
>
> Because a message can be delivered more than once — that is the price of guaranteed delivery — your workers **must be idempotent**. Processing the same message twice must not charge the card twice. Remember §3.2? This is why that concept matters.

6.5 Where "serverless" fits

A traditional server sits in a loop waiting whether or not requests arrive, and you pay for that waiting. Serverless flips it: your code exists only while handling something. **Azure Functions** runs one function when an HTTP request arrives, a queue message appears, or a timer fires — you pay per execution and per GB-second, and idle cost is zero. **Azure Container Apps** scales your container from zero based on HTTP traffic or queue depth.

The name is a marketing lie — there are absolutely servers — but the *developer experience* has none: no OS patching, no capacity planning, no idle billing. **The catch is cold starts:** if nothing has run for a while, the first request waits for an instance to spin up (hundreds of milliseconds to seconds). Fine for a background job, potentially unacceptable for a checkout page. Mitigations exist and they cost money — which is the trade-off in a nutshell.

7. Web Servers: Apache, Nginx, IIS

A web server listens on a port, accepts HTTP requests, and returns HTTP responses. In practice it does five things: **serve static files** (map a URL to a file and send its bytes with the right `Content-Type`); **route** (decide which site and handler a request belongs to); **terminate TLS** (do the crypto so your app does not have to); **proxy** (forward to an application elsewhere and relay the answer); and **enforce policy** (compression, caching, rate limits, redirects, request size limits).

Static content is identical for everybody — a logo, a CSS file, a PDF. The server reads a file and sends it, tens of thousands of times per second, and a CDN can cache it near the user. **Dynamic content** is generated per request and requires running application code, which is orders of magnitude more expensive. *The rule that follows: serve static content statically.* Do not route `/logo.png` through your Django application.

7.1 The three you will meet

	Apache HTTP Server	Nginx	IIS
Released / runs on	1995 · Linux, Windows, macOS	2004 · Linux, macOS	1995 · Windows only
Concurrency	Process/thread per connection	Event-driven, async	Thread pool + async I/O
Under heavy load	Memory grows with connections	Stays flat and low	Good on Windows
Config	`httpd.conf` + `.htaccess`	`nginx.conf`, nested blocks	`web.config` (XML) + GUI
Killer feature	Per-folder config with no restart; huge module ecosystem	Speed as reverse proxy and static host	Deep .NET, Windows auth, AD integration
Typical role today	Legacy PHP/WordPress	Reverse proxy / load balancer — **the default choice**	ASP.NET on Windows

The concurrency difference, simply. Apache's classic model dedicates a worker to each connection, each using a few megabytes of RAM; 10,000 slow mobile connections means 10,000 workers and the machine falls over. Nginx uses a small fixed number of workers, each juggling thousands of connections with an event loop — one waiter efficiently managing forty tables instead of hiring forty waiters. (Modern Apache with `mpm_event` narrows the gap considerably, but the default choice stuck.) **Learn Nginx.**

7.2 Virtual hosts — many sites, one server

One VM, one IP, and you want `shop.com`, `blog.com`, and `portfolio.com` on it. The answer is the `Host` header (§3.4): every HTTP/1.1 request carries the hostname the client typed, and the server picks the matching

site config. Apache calls these *Virtual Hosts*, Nginx calls them *server blocks*, IIS calls them *Sites* — same concept.

```
server {
    listen 80;
    server_name shop.com www.shop.com;
    root /var/www/shop;
    index index.html;
}
server {
    listen 80;
    server_name blog.com www.blog.com;
    root /var/www/blog;
}
```

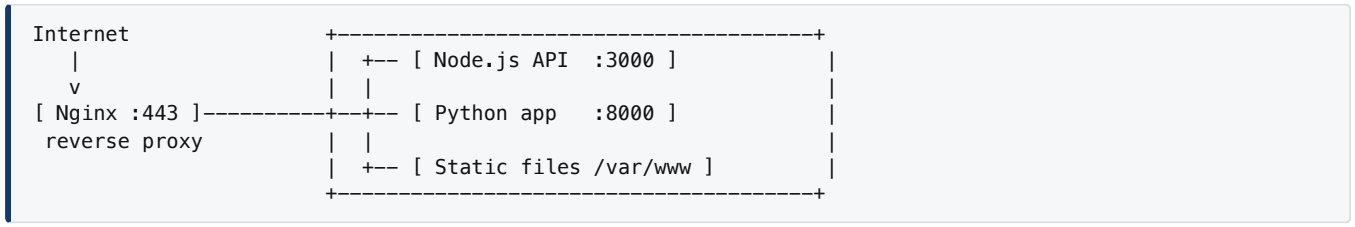
> **THE HTTPS CHICKEN-AND-EGG**
> With encryption the `Host` header is *inside* the encrypted payload, but the server must choose a certificate *before* decryption begins. The fix is **SNI (Server Name Indication)**, a TLS extension where the client announces the hostname in the clear during the handshake. Every browser since roughly 2010 supports it. In Azure App Service, "SNI SSL" is the standard free binding; "IP-based SSL" dedicates an IP and costs extra.

In App Service you do not edit virtual host files — you add custom domains and Azure's front-end fleet routes:

```
az webapp config hostname add --webapp-name myApp --resource-group myRG --hostname www.example.com
```

7.3 Reverse proxy — the concept that unlocks everything else

This is the most important architectural idea in this module and the one students find most slippery. **A reverse proxy receives requests on behalf of other servers, forwards them, and relays the answers back.** The client thinks it is talking to the proxy and has no idea anything else exists.



	Forward proxy	Reverse proxy
Sits in front of	**Clients**	**Servers**
Hides	Who the client is	What the servers are
Works for	The user	The website owner
Example	Corporate web filter, VPN, Tor	Nginx in front of your app, Azure Front Door

Same technology, opposite direction, opposite beneficiary. **Why put one in front of your app?**

1. **Port privilege.** Your Node app runs unprivileged on 3000; Nginx handles 443. Your application never runs as root.
2. **TLS termination.** One place to renew certificates; your app speaks plain HTTP internally.
3. **One entry point, many apps.** `/api` → Node, `/admin` → Python, rest → static. One domain, one certificate.
4. **Load balancing** across instances, with health checks.
5. **Static file offloading** — your app process never wastes a thread on a logo.

6. **Buffering slow clients.** Nginx accepts a slow 3G upload at its own pace and hands your app the complete request at once, so an expensive worker is not tied up for 40 seconds.
7. **Security.** Rate limiting, size limits, hiding version headers, WAF rules — all before a byte reaches your code.
8. **Zero-downtime deploys.** Start the new version on a new port, flip the proxy, retire the old one.

```
server {
    listen 443 ssl;
    server_name api.example.com;
    ssl_certificate     /etc/letsencrypt/live/api.example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/api.example.com/privkey.pem;

    location / {
        proxy_pass http://127.0.0.1:3000;

        # Pass the ORIGINAL request details. Without these, your app thinks
        # every request came from 127.0.0.1 over plain HTTP.
        proxy_set_header Host              $host;
        proxy_set_header X-Real-IP         $remote_addr;
        proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        # Allow WebSocket upgrades through
        proxy_http_version 1.1;
        proxy_set_header Upgrade    $http_upgrade;
        proxy_set_header Connection "upgrade";

        proxy_read_timeout 60s;      # give up after 60s -> produces a 504
    }

    location /static/ {             # served directly, never touching the app
        alias /var/www/static/;
        expires 30d;
    }
}
```

> **X-FORWARDED-FOR MATTERS MORE THAN IT LOOKS**
>
> Once a proxy is in the path, your application's "client IP" is the proxy. Any logging, rate limiting, or geo-blocking based on IP will be *wrong* unless you read `X-Forwarded-For`. App Service and Front Door set these automatically; most frameworks have a "trust proxy" setting you must enable.

7.4 Where Azure puts all of this

You will rarely install Nginx by hand in this course — Azure has already done it. It is the same machinery underneath, which is why you needed to understand it.

Concept	Azure equivalent
Virtual host	Custom domain on an App Service
Reverse proxy	Built into App Service; also Application Gateway / Front Door
TLS termination	App Service Managed Certificate, or Front Door
Load balancer	App Service scale-out, Load Balancer, App Gateway
Static file host	Azure Storage static website + CDN
WAF	Application Gateway WAF / Front Door WAF

When you deploy a Python app to App Service, Azure runs it behind a proxy that terminates TLS and forwards to an internal port. Seeing `X-Forwarded-Proto: https` in your app's headers is this machinery doing its job.

8. Configuration Files and Logs

Web servers are general-purpose programs; a config file tells one which job it is doing today, and logs tell you what actually happened. Configuration is where you will spend the frustrating hours. Logs are how you get them back.

8.1 nginx.conf

Nginx config is a tree of **blocks** (contexts) containing **directives** ending with semicolons. Settings cascade downward: `gzip on;` in `http` applies to every `server` unless overridden — which is why the same directive can appear at several levels.

```
http {
    include       /etc/nginx/mime.types;   # maps .css -> text/css, etc.
    access_log    /var/log/nginx/access.log main;
    client_max_body_size 20M;              # reject bigger uploads with 413
    gzip on;

    server {                               # ---- one website ----
        listen       80;
        server_name  example.com www.example.com;
        root         /var/www/html;
        index        index.html;

        location / {                       # ---- one URL pattern ----
            try_files $uri $uri/ =404;
        }
        location /api/ {
            proxy_pass http://127.0.0.1:3000/;
        }
        location ~* \.(jpg|png|css|js|woff2)$ {
            expires 30d;
            access_log off;                # don't log every image
        }
        error_page 500 502 503 504 /50x.html;
    }
}
```

Syntax	Meaning	Priority
`location = /exact`	Exact match only	1st — wins immediately
`location ^~ /images/`	Prefix, stop looking at regex	2nd
`location ~ \.php$` / `~*`	Regex, case-sensitive / insensitive	3rd — first match wins
`location /docs/`	Plain prefix	4th — longest match wins

THE TWO COMMANDS YOU MUST KNOW

```
sudo nginx -t                 # TEST the config. ALWAYS run this first.
sudo systemctl reload nginx   # apply WITHOUT dropping connections
```

Reloading a broken config is how you take a site down at 2 a.m. `nginx -t` takes half a second. Run it every time.

8.2 .htaccess (Apache) and web.config (IIS)

`.htaccess` is per-directory config that Apache reads on **every request**. That means you can change behaviour without a restart and without root — which is why every shared host relies on it — but it is slow and errors

only surface at request time. If you control the server, put the rules in the main config and set `AllowOverride None`.

```
# Force HTTPS
RewriteEngine On
RewriteCond %{HTTPS} off
RewriteRule ^(.*)$ https://%{HTTP_HOST}%{REQUEST_URI} [L,R=301]

# Send anything that isn't a real file to index.php (front controller)
RewriteCond %{REQUEST_FILENAME} !-f
RewriteRule ^ index.php [L]

<Files ".env">
    Require all denied
</Files>
```

`R=301` is the status code from §3.3, right there in a config file. IIS expresses the same concepts in XML, and this is directly course-relevant: a **Windows-based Azure App Service reads `web.config` from the root of your deployed site.**

```
<configuration><system.webServer>
  <defaultDocument><files><clear /><add value="index.html" /></files></defaultDocument>
  <rewrite><rules>
    <rule name="Force HTTPS" stopProcessing="true">
      <match url="(.*)" />
      <conditions><add input="{HTTPS}" pattern="off" ignoreCase="true" /></conditions>
      <action type="Redirect" url="https://{HTTP_HOST}/{R:1}" redirectType="Permanent" />
    </rule>
  </rules></rewrite>
  <httpProtocol><customHeaders><remove name="X-Powered-By" /></customHeaders></httpProtocol>
</system.webServer></configuration>
```

8.3 Reading logs

Two kinds, and knowing which to open saves an hour. The **access log** has one line per request and answers *"did the request arrive, and what did we return?"* The **error log** has one line per problem and answers *"why did it fail?"*

```
203.0.113.9 - - [27/Aug/2026:09:14:22 +0500] "GET /api/students?page=2 HTTP/1.1" 200 4821
  client IP           timestamp                  request line                status size
```

```
2026/08/27 09:14:22 [error] *1029 connect() failed (111: Connection refused)
while connecting to upstream, client: 203.0.113.9, server: api.example.com,
upstream: "http://127.0.0.1:3000/api/students"
```

Read that carefully — it contains the complete answer. `connect() failed (111: Connection refused)` *while connecting to upstream* `http://127.0.0.1:3000`. Translation: **Nginx tried to reach your app on port 3000 and nothing was listening.** Your app is not running. That is the whole diagnosis, handed to you.

Platform	Access log	Error log
Nginx (Linux)	`/var/log/nginx/access.log`	`/var/log/nginx/error.log`
Apache (Debian/Ubuntu)	`/var/log/apache2/access.log`	`/var/log/apache2/error.log`
IIS	`C:\inetpub\logs\LogFiles\`	Windows Event Viewer
Azure App Service	Kudu → LogFiles, or `az webapp log tail`	

```
tail -f /var/log/nginx/access.log                    # watch requests live
awk '{print $9}' access.log | sort | uniq -c | sort -rn  # count status codes
grep " 50[0-9] " access.log                          # every 5xx today
awk '{print $1}' access.log | sort | uniq -c | sort -rn | head   # top client IPs
az webapp log tail --name myApp --resource-group myRG  # stream Azure logs
```

8.4 Case study: debugging a 502 Bad Gateway

`502` deserves its own walkthrough because it is the error you will hit most in cloud deployments and the message itself is useless. It means: *"I am a proxy. I forwarded your request to the server behind me. What came back was nothing, or garbage I could not parse."*

> **CRITICALLY**
>
> **502 is not your app returning an error.** If your app returned a 500, the proxy would faithfully pass the 500 along. A 502 means the proxy could not get a valid HTTP response *at all*.

#	Cause	Log signature	Fix
1	**App is not running** (crashed at startup)	`connect() failed (111: Connection refused)`	Check app logs — usually a syntax error, missing env var, or failed DB connection at boot
2	**Wrong port** — proxy points at 3000, app listens on 8080	`Connection refused` on an unused port	Align `proxy_pass` with the app's actual port
3	**Bound to `127.0.0.1`** in a container	`Connection refused`	Bind `0.0.0.0`. **The #1 cause on App Service and Container Apps.**
4	App too slow	`upstream timed out`	Often surfaces as 504; raise `proxy_read_timeout` and fix the slow query
5	App crashed mid-request (OOM)	`upstream prematurely closed connection`	Check memory limits, look for OOM-killer messages
6	Response headers too large (giant cookie/JWT)	`upstream sent too big header`	Increase `proxy_buffer_size`, or shrink the token
7	TLS mismatch — proxy speaks HTTPS to an HTTP-only backend	SSL handshake errors	Use `proxy_pass http://` for internal hops

```
# Diagnostic procedure - in this order
ps aux | grep node                    # 1. Is the process alive at all?
sudo ss -tulpn | grep 3000            # 2. Is anything listening?
   #  nothing at all?        -> cause #1
   #  shows 127.0.0.1:3000?  -> cause #3
   #  shows 0.0.0.0:3000?    -> app is fine, keep going
curl -v http://127.0.0.1:3000/health   # 3. Can it answer, bypassing the proxy?
   #  works? -> the proxy config is the problem
   #  fails? -> the app is the problem
tail -50 /var/log/nginx/error.log      # 4. What did the proxy try to do?
az webapp log tail -n myApp -g myRG    # 5. On Azure App Service
```

> **THE APP SERVICE 502 THAT CATCHES EVERYONE**
> App Service injects the port your app must listen on via the `PORT` environment variable and expects your app to honour it. Hardcoding `app.listen(3000)` produces an instant 502. Write `app.listen(process.env.PORT || 3000)`, or in Python `port = int(os.environ.get("PORT", 8000))`. Put this on a sticky note.

At scale, text logs stop working. Structured logging emits JSON instead, so you can *query* logs rather than read them. Azure collects these into **Log Analytics**, queried with KQL, and **Application Insights** traces a single request across every service it touches — the industrial version of `tail -f`.

```
AppServiceHTTPLogs
| where TimeGenerated > ago(1h)
| where ScStatus >= 500
| summarize count() by CsUriStem, ScStatus
| order by count_ desc
```

9. SSL/TLS and HTTPS in Practice

Over plain HTTP, your request travels as readable text through every device between you and the server: your router, your ISP, the café Wi-Fi access point, several backbone routers. **Any of them can read it. Any of them can change it.**

Threat	What happens
Eavesdropping	Someone reads your password, session cookie, private messages.
Tampering	Someone modifies the response — injecting ads or malware, altering a transfer's account number.
Impersonation	Someone pretends to *be* the server. You type your password into a perfect replica.

Encryption alone solves the first two. It does **not** solve the third — an encrypted connection to an attacker is still a connection to an attacker. That is why certificates exist. TLS provides **confidentiality** (symmetric AES encryption), **integrity** (message authentication codes), and **authentication** (certificates and public-key cryptography). SSL is its dead predecessor; everyone still says "SSL certificate" out of habit, but the protocol running is TLS 1.2 or 1.3.

> **WHAT THE PADLOCK DOES NOT MEAN**
> TLS says nothing about whether a site is *trustworthy*. A phishing site can have a perfectly valid certificate. The padlock means "you are securely connected to whoever owns this domain name" — not "this business is honest". Teach that to your relatives.

9.1 Certificates and the chain of trust

Anyone can generate a key pair. How do you know the key you received belongs to `lums.edu.pk` and not to somebody sitting between you and it? A **Certificate Authority** verifies that whoever asked really controls the domain, then digitally signs a document saying so. That document is the certificate.

The analogy is your passport: you could forge a document claiming to be you, but a border officer trusts your passport because it was issued by a government whose signature they recognise. You need not know the officer; you both trust the same issuer.

```
[ Root CA certificate ]        pre-installed in your OS/browser (~150 of them),
        | signs                kept offline in vaults, never touches your traffic
        v
[ Intermediate CA cert ]       the day-to-day workhorse
        | signs                YOUR SERVER MUST SEND THIS ONE TOO
        v
[ Your server certificate ]    "lums.edu.pk", valid 90 days
```

Your browser walks up that chain. Arrive at a trusted root and the connection is trusted; break the chain and you get `NET::ERR_CERT_AUTHORITY_INVALID`.

> **THE #1 CERTIFICATE INSTALLATION MISTAKE**
> Installing only your own certificate and forgetting the intermediate. Desktop Chrome often papers over it by fetching the missing link, so *it works on your laptop* — but mobile apps, `curl`, and Java clients fail. This is why Nginx wants `fullchain.pem` (your cert + intermediates), not `cert.pem`. Verify with `openssl s_client -connect example.com:443 -showcerts` or ssllabs.com/ssltest.

By validation depth: DV (domain validated — minutes, automated, free) proves you control the domain; OV and EV add organisational and legal vetting for days/weeks and real money. Browsers stopped showing the green company name for EV years ago because studies showed nobody noticed. **For nearly everything, DV is correct and free. By coverage:** single domain; wildcard `*.example.com` (covers `api.`, `www.` — but **not** `a.b.example.com`); or SAN/multi-domain lists.

9.2 The handshake, without the mathematics

```
CLIENT                                                    SERVER
 1. ClientHello: "TLS 1.3, these ciphers,      ---->
    and I want 'api.example.com'" (SNI)
                                               <----   2. ServerHello + certificate chain
 3. Client verifies: signed by a CA I trust? not expired?
    domain matches what I asked for? not revoked?
 4. Key exchange - both sides derive the SAME  <--->
    symmetric key without ever sending it (Diffie-Hellman)
 5. Encrypted HTTP traffic from here on        <--->
```

Two things worth internalising. **Asymmetric crypto is used only to establish trust and agree a key**; bulk data is then encrypted with fast symmetric crypto, because public-key operations per byte would be unusable. And **the session key is never transmitted** — both sides compute it independently from exchanged public values, so an eavesdropper recording the entire handshake still cannot derive it. Because a fresh key is generated per session (*forward secrecy*), stealing the server's private key next year does not decrypt traffic recorded today.

9.3 Binding a certificate

"Binding" means telling the server which certificate to present for which hostname on which port.

```
server {
    listen 443 ssl http2;
    server_name example.com www.example.com;

    ssl_certificate     /etc/letsencrypt/live/example.com/fullchain.pem;  # cert + intermediates
    ssl_certificate_key /etc/letsencrypt/live/example.com/privkey.pem;    # NEVER share this

    ssl_protocols       TLSv1.2 TLSv1.3;   # 1.0 and 1.1 are deprecated - do not enable
    ssl_session_cache   shared:SSL:10m;    # resume sessions, skip full handshakes
}
```

Extension	Contains	Used by
`.pem` / `.crt` / `.cer`	Base64 text, `-----BEGIN CERTIFICATE-----`	Nginx, Apache, Linux tooling
`.key`	The **private key**. Guard like a password.	Nginx, Apache
`.pfx` / `.p12`	Certificate **and** key in one password-protected binary	**IIS and Azure App Service**

```
openssl pkcs12 -export -out mycert.pfx -inkey privkey.pem -in fullchain.pem   # PEM -> PFX
```

9.4 Getting a certificate

Let's Encrypt — free, automated, 90-day certificates. The short lifetime is deliberate: it forces automation, and automated renewal is how you stop having outages on a public holiday. **Azure App Service Managed Certificate** — free, auto-renewing, zero configuration, and the right default for this course.

```
sudo certbot --nginx -d example.com -d www.example.com   # edits config, installs, schedules
renewal
sudo certbot renew --dry-run                             # verify auto-renewal works

az webapp config hostname add -g myRG --webapp-name myApp --hostname www.example.com
az webapp config ssl create   -g myRG --name myApp --hostname www.example.com
az webapp config ssl bind     -g myRG --name myApp \
    --certificate-thumbprint <THUMBPRINT> --ssl-type SNI    # SNI is free; IP-based costs extra
```

Certificates for production apps you own should live in **Azure Key Vault** — centralised, access-controlled, audited, and importable directly into App Service and Application Gateway.

9.5 Forcing HTTPS

Having a certificate is not enough if port 80 still serves the site. Two layers:

```
# Layer 1 - redirect. Nginx:
server { listen 80; server_name example.com; return 301 https://$host$request_uri; }

# Azure App Service - one flag, no code:
az webapp update --resource-group myRG --name myApp --https-only true

# Layer 2 - HSTS:
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
```

A redirect still involves one unencrypted round trip, which an attacker on the network can hijack (an "SSL stripping" attack). **HSTS** closes that gap: the server tells the browser *"for the next year, refuse to talk to me over HTTP at all"*. After the first successful HTTPS visit, the browser rewrites `http://` internally before any packet leaves the machine.

> **HSTS IS A COMMITMENT, NOT A SETTING**
> Once a browser has cached it you *cannot* serve that domain over HTTP until the max-age expires, and you cannot cancel it remotely. If you set `includeSubDomains` and later need an internal subdomain on plain HTTP, you are stuck. Test with a short `max-age` (e.g. 300) first, then raise it.

Mixed content: if your HTTPS page loads a script over `http://`, browsers block it outright and your page breaks silently. The DevTools console names the guilty resource.

10. File Transfer: FTP, FTPS, SFTP

You have built a website. The files are on your laptop; the server is in a datacentre. Something must move the bytes. Historically that was FTP. Today FTP is a security liability, and understanding *why* teaches something general about protocols.

FTP dates from 1971 — older than TCP/IP, older than email as you know it — designed for a research network where everyone knew everyone. **The fatal flaw: everything is plaintext.** Username, password, and every byte of every file travel unencrypted; anyone on the same network can capture credentials with a tool that takes thirty seconds to set up. **The second flaw: two connections.** FTP separates control from data, which makes it miserable for firewalls and NAT — *active* mode has the server connect back to the client (client firewalls block this; it almost always fails today), and *passive* mode works from behind NAT but requires a wide range of server ports to be open.

10.1 FTPS vs SFTP — not related, despite the names

	FTPS	**SFTP**
Full name	FTP **S**ecure	**SSH** File Transfer Protocol
Built on	Old FTP + a TLS wrapper	SSH — a completely different modern protocol
Relation to FTP	It **is** FTP, encrypted	**Nothing to do with FTP at all**
Ports	21 (explicit) or 990 (implicit) + a data port range	**22 only** — one connection, one port
Firewall friendliness	Awkward (port ranges)	Easy
Authentication	Username/password + certificates	Password **or SSH keys** (much stronger)
Verdict	Acceptable; used by Azure App Service	**Preferred wherever available**

In one line: FTPS is old FTP wearing a TLS coat; SFTP is a file-transfer feature of SSH that happens to have a confusingly similar name. (*Explicit* FTPS starts on port 21 in the clear and issues `AUTH TLS` to upgrade — the modern default, and what App Service uses. *Implicit* assumes encryption from byte one on port 990 and is deprecated.)

10.2 SSH keys — do this instead of passwords

A key pair is two files: a **private key** you keep forever and never send anywhere, and a **public key** you copy onto every server you want to reach. The server challenges you to prove you hold the private key; you never transmit a secret.

```
ssh-keygen -t ed25519 -C "27100431@lums.edu.pk"
   # -> ~/.ssh/id_ed25519       PRIVATE. Never leaves your machine. Never commit to Git.
   # -> ~/.ssh/id_ed25519.pub   PUBLIC. Safe to share, paste anywhere.

ssh-copy-id azureuser@20.51.3.44         # install the public key on a server
sftp azureuser@20.51.3.44                # transfer over the same secure channel
scp ./index.html azureuser@20.51.3.44:/var/www/html/
rsync -avz ./site/ azureuser@20.51.3.44:/var/www/html/    # only copies what changed
```

Azure Linux VMs are created with SSH key authentication by default and password login disabled — good defaults, and now you know why.

10.3 What professionals actually use

Manual file transfer is an anti-pattern for application deployment. If a human drags files onto a server, the server's state depends on that human's memory and nobody can tell you what version is running.

Method	Use for
Git push / GitHub Actions / Azure Pipelines	Application deployment. Reproducible, reviewable, revertable.
Container images (ACR → App Service / AKS)	The whole environment, versioned as one artifact.
`az webapp deploy` / zip deploy	Scripted deployment of a build output.
Blob Storage + `azcopy`	Bulk data, backups, media, static sites.
SFTP	Genuine ad-hoc movement, or partner data exchange.
FTPS	Legacy systems and quick App Service fixes.
Plain FTP	**Never.**

```
az webapp config set -n myApp -g myRG --ftps-state FtpsOnly   # reject plain FTP
az webapp config set -n myApp -g myRG --ftps-state Disabled   # best for CI/CD-deployed apps
```

Set `Disabled` on anything deployed through a pipeline. An unused open door is still an open door, and App Service FTP credentials are a common security-audit finding.

11. Bringing It to Azure

Term	Meaning
Subscription	The billing container. Everything you create costs money against one.
Resource Group	A folder for related resources. Delete the group, delete everything inside. **Use one per project** — the cleanest way to avoid a surprise bill.
Region	Physical location (`eastus` , `uaenorth`). Affects latency, cost, and data residency law.
App Service Plan	The *hardware* — the VMs your apps run on. Sized by SKU (F1 free, B1 basic, P1v3 production).
Web App	The *application* running on a plan. Many Web Apps can share one plan.

> **THE RELATIONSHIP THAT CONFUSES EVERYONE**
>
> You pay for the **App Service Plan**, not the Web App. The plan is the rented apartment; Web Apps are furniture you put inside it. Ten Web Apps on one B1 plan cost the same as one — until they exhaust the CPU.

11.1 Azure App Service

App Service is a **managed web server**. Microsoft runs the OS, patching, reverse proxy, load balancer, and TLS termination; you supply code. Every concept from §7–§9 is present — you just do not administer it.

```
Internet  https://myapp.azurewebsites.net
   v
[ Azure front-end fleet ]  TLS termination (§9), load balancing (§6.3),
   v                       custom domain routing (§7.2)
[ Your app, on the PORT env var ]   <- this part is yours
   v
[ Azure SQL / Blob / Redis ]
```

You get HTTPS on `*.azurewebsites.net` out of the box, auto-scaling on paid tiers, **deployment slots** (a full staging copy you warm up then *swap* into production with no downtime), built-in Application Insights and log streaming, and managed identity so your app reaches Key Vault and SQL without stored passwords.

```
az login
az group create --name cs487-rg --location eastus
az appservice plan create --name cs487-plan -g cs487-rg --sku B1 --is-linux
az webapp create --name cs487-demo-27100431 -g cs487-rg --plan cs487-plan --runtime "PYTHON:3.11"
az webapp up      --name cs487-demo-27100431 -g cs487-rg      # deploy current folder
az webapp log tail --name cs487-demo-27100431 -g cs487-rg     # watch it start
az webapp browse   --name cs487-demo-27100431 -g cs487-rg
```

Never hardcode configuration. App settings arrive in your app as environment variables, read with `os.environ["API_KEY"]` or `process.env.API_KEY`. This is how one code artifact runs in dev, staging, and production unmodified — and how secrets stay out of your Git history.

```
az webapp config appsettings set -n myApp -g cs487-rg \
  --settings DATABASE_URL="..." API_KEY="..." ENVIRONMENT="production"
```

The settings that map directly onto this module:

```
az webapp update     -g cs487-rg -n myApp --https-only true               # §9.5
az webapp update     -g cs487-rg -n myApp --client-affinity-enabled false # §6.2
az webapp config set -g cs487-rg -n myApp --ftps-state FtpsOnly           # §10.3
az webapp config set -g cs487-rg -n myApp --min-tls-version 1.2           # §9.3
az webapp cors add   -g cs487-rg -n myApp --allowed-origins https://mysite.com  # §3.5
az webapp log config -g cs487-rg -n myApp \
    --web-server-logging filesystem --application-logging filesystem --level information
```

Kudu — every App Service has a hidden admin site at `https://<app-name>.scm.azurewebsites.net` giving you a file browser, a shell, environment variables, a process explorer, and raw logs. When a deployment behaves strangely, look there first.

11.2 Static website hosting on Azure Storage

If your site is purely static — HTML, CSS, JS, images, no server-side code — you do not need a web server at all. Azure Storage serves the files directly for cents per month at effectively unlimited scale. That covers more than you would think: React/Vue/Angular SPAs, documentation sites, portfolios, Hugo/Jekyll blogs, and any frontend that calls an API for its data.

```
az storage account create --name cs487static27100431 -g cs487-rg \
    --location eastus --sku Standard_LRS --kind StorageV2

az storage blob service-properties update --account-name cs487static27100431 \
    --static-website --index-document index.html --404-document 404.html
    # creates a special container literally named  $web

az storage blob upload-batch --account-name cs487static27100431 \
    --source ./dist --destination '$web' --overwrite

az storage account show --name cs487static27100431 -g cs487-rg \
    --query "primaryEndpoints.web" --output tsv
```

> **A CLASSIC FIVE-MINUTE TIME-WASTER**
> In bash and zsh, quote `'$web'` in single quotes or the shell expands it to an empty variable and your upload lands in the wrong place.

	Storage static website	App Service	Static Web Apps
Runs server code	No	Yes	Via Azure Functions
Cost	Pennies	~$13+/mo (B1)	Free tier available
Custom domain + TLS	Needs CDN/Front Door	Built in	Built in
Best for	Bulk static assets, SPAs	APIs, server-rendered apps	Modern SPA + API together

11.3 A complete reference architecture

```
                        [ Users ]
                           |  HTTPS (§9)
                  [ Azure Front Door ]  global routing, CDN, WAF, TLS
              +-------------+--------------+
              v                            v
   [ Storage static website ]     [ App Service (API, stateless) ]   §6.2, §7.3
   [ $web: HTML/CSS/JS      ]     [ auto-scaled 2..10            ]
                                     |        |          |
                                     v        v          v
                               [ Azure SQL ] [ Redis ] [ Service Bus ]   §6.4
                                  data      sessions     job queue
                                                            |
                                                            v
                                                   [ Container Apps ]
                                                    workers, scale to zero
```

Trace one request through that diagram and name what happens at each hop. If you can do that out loud, you have understood this module.

12. Hands-On Labs

Lab 1 — Read the web (30 min, no installation)

Open lums.edu.pk with DevTools → Network and reload. From the panel, answer: how many requests did one page load produce? What is the `Content-Type` of the first (document) response? Find a request that returned `304` — why did that save bandwidth? What TLS version was used (Security tab)? Then run `curl -I https://www.lums.edu.pk`, `curl -i https://httpbin.org/status/404`, and `curl -s https://httpbin.org/headers`.

Deliverable: a table of five requests — URL, method, status, content type, size — with one sentence each on why that status was returned.

Lab 2 — Talk to an API (45 min)

```
curl -s https://api.github.com/users/octocat | python -m json.tool
curl -X POST https://httpbin.org/post -H "Content-Type: application/json" \
     -d '{"rollNo":"27100431","course":"CS487"}'
curl -X POST https://httpbin.org/post -H "Content-Type: application/json" \
     -d "{'bad': 'quotes',}"                          # deliberately broken - read the error
curl -s https://httpbin.org/bearer -H "Authorization: Bearer test123"
```

Deliverable: design a REST API for a **library** with resources `books`, `members`, and `loans`. Give the full method + path table (at least 10 endpoints), one example request body, one success response, and one error response in the shape shown in §4.3.

Lab 3 — Run a web server and break it on purpose (60 min)

```
mkdir -p site && echo "<h1>Hello from CS 487</h1>" > site/index.html
docker run -d --name web -p 8080:80 -v "$(pwd)/site":/usr/share/nginx/html:ro nginx
curl -i http://localhost:8080
docker logs web                              # your access log
docker exec -it web cat /etc/nginx/nginx.conf
```

Then: (1) change the port mapping to 8081 and confirm 8080 stops working; (2) add a second virtual host and test it with `curl -H "Host: site2.local" http://localhost:8080`; (3) delete a semicolon and run `nginx -t`, reading the error; (4) add `location /api/ { proxy_pass http://127.0.0.1:3000/; }` with **nothing** running on 3000, then request it. **You now have a 502 you created yourself.** Find it in the error log and identify which of the seven causes in §8.4 it is.

Deliverable: screenshots of the 502, the matching error log line, and a one-paragraph diagnosis.

Lab 4 — Deploy to App Service, then prove statelessness (90 min)

```
# app.py
import os
from flask import Flask, jsonify
app = Flask(__name__)

@app.get("/")
def home():    return "<h1>CS 487 - deployed to Azure</h1>"

@app.get("/health")
def health():  return jsonify(status="ok")

if __name__ == "__main__":
    # Read the port from the environment - see the §8.4 warning
    app.run(host="0.0.0.0", port=int(os.environ.get("PORT", 8000)))
```

Deploy with `az webapp up` (§11.1) and confirm `/health` returns JSON. Then turn on `--https-only` and verify `curl -I http://...` issues a `301`; add an app setting and read it in code; stream logs with `az webapp log tail` while hitting the site; find your files in Kudu. **Then deliberately break it:** hardcode `port=3000`, redeploy, observe the 502, and fix it.

Stretch — make §6.2 visible. Add an in-memory counter at `/count`, scale to 3 instances with affinity off, hit it twenty times, and watch the number jump around unpredictably. Explain in writing why, and describe how Azure Cache for Redis would fix it.

Deliverable: the live URL, a screenshot of the 301, the log excerpt showing the 502 you caused and the line that fixed it, and the counter explanation.

13. Troubleshooting Quick Reference

What you see	Most likely cause	First thing to check
`Connection refused`	Nothing listening on that port	`ss -tulpn | grep <port>`
`Connection timed out`	A firewall/NSG silently dropping packets	NSG rules, then OS firewall
`ERR_NAME_NOT_RESOLVED`	DNS failure	`nslookup yourdomain.com`
`400`	Malformed body or missing field	Validate your JSON
`401` / `403`	Missing/expired token · authenticated but not permitted	Is `Authorization` actually being sent? Then role assignment
`404`	Wrong path, or wrong `root` / `DocumentRoot`	Does the file exist where the config says?
`413`	Upload exceeds proxy limit	`client_max_body_size`
`500`	Unhandled exception in **your** code	Application log / stack trace
`502`	Proxy cannot reach the app	§8.4 — usually not running, or bound to `127.0.0.1`
`504`	App too slow	Find the slow query; raise timeouts as a stopgap
`NET::ERR_CERT_*`	Expired / wrong domain / missing intermediate	`openssl s_client -showcerts`
Works in curl, fails in browser	**CORS**	§3.5 — check `Access-Control-Allow-Origin`
Works locally, 502 on Azure	Hardcoded port	Use `process.env.PORT` / `os.environ["PORT"]`
Works on the VM, not from outside	Bound to `127.0.0.1`	Bind `0.0.0.0`, then check the NSG
Users randomly logged out	Stateful app across multiple instances	§6.2 — externalise session state
`EADDRINUSE`	Port already taken	`lsof -i :<port>` and kill it

UNIVERSAL DIAGNOSTIC LADDER

Work outward from the app, and do not skip steps — the answer is usually two rungs below where you started guessing.

```
1. Is the process running?           ps aux | grep <app>
2. Is it listening, and on what?     ss -tulpn
3. Does it answer locally?           curl -v http://127.0.0.1:<port>/health
4. Does the proxy reach it?          tail -f /var/log/nginx/error.log
5. Is the port open at the network?  NSG rules, OS firewall
6. Does DNS point at the right IP?   nslookup / dig
7. Is the certificate valid?         openssl s_client -connect host:443
8. What did the client actually send? Browser DevTools -> Network
```

14. Self-Check

Answer in your own words before reading on. These are the eight that matter most.

1. What actually distinguishes 401 from 403?

401 means *unauthenticated* — you have not proven who you are. 403 means *unauthorised* — your identity is established but lacks permission.

2. **Why is it dangerous to expose a destructive action over GET?**
   GET is defined as *safe*, so browsers pre-fetch it, proxies cache it, and crawlers follow it automatically. A `GET /deleteUser?id=5` link will eventually be triggered by a bot with no human involved.

3. **What does idempotency buy you in a distributed system?**
   Safe retries. A timeout does not tell you whether the server processed the request. If the operation is idempotent you simply retry; if not, retrying risks a double charge or duplicate order.

4. **One IP, three HTTPS sites. How does the server pick the right certificate?**
   SNI. The client sends the hostname in plaintext during the ClientHello, before encryption begins, so the server can select the matching certificate.

5. **Why is statelessness a prerequisite for elastic scaling?**
   Auto-scaling adds and destroys instances continuously. If an instance holds a session in memory, destroying it destroys the session, and a new instance cannot serve requests assuming prior context. Stateless instances are interchangeable.

6. **Give three reasons to run Nginx in front of a Node app.**
   Any three of: TLS termination in one place; the app avoids running as root on 443; static files served without touching the app process; buffering slow clients; load balancing; rate and size limits before code executes; zero-downtime deploys by switching upstreams.

7. **Why is the intermediate certificate needed, and why does forgetting it still "work on my laptop"?**
   Clients must build an unbroken chain to a trusted root. Desktop browsers often fetch a missing intermediate automatically, masking the problem — but curl, mobile apps, and Java clients do not, and they fail. Serve `fullchain.pem`.

8. **Your app runs locally but returns 502 on App Service. Most likely cause?**
   A hardcoded port. App Service tells your app which port to bind via `PORT` and routes only there. `app.listen(3000)` gives an instant 502; `app.listen(process.env.PORT || 3000)` fixes it. Related: binding `127.0.0.1` instead of `0.0.0.0`.

15. Further Reading

- **HTTP** — MDN Web Docs: `developer.mozilla.org/en-US/docs/Web/HTTP` (status codes and CORS sections especially)
- **Nginx** — beginner's guide: `nginx.org/en/docs/beginners_guide.html`
- **TLS** — Mozilla SSL config generator (`ssl-config.mozilla.org`) generates a hardened block for you; SSL Labs (`ssllabs.com/ssltest`) grades any live site A+ to F and explains every deduction — the fastest TLS education available
- **Azure** — App Service overview and configuration: `learn.microsoft.com/azure/app-service/` ; static websites: `learn.microsoft.com/azure/storage/blobs/storage-blob-static-website`
- **Practice tools** — `httpbin.org` (request sandbox), `jsonlint.com` (validator), `jwt.io` (decode a token)
- **Certification alignment** — this module supports material assessed in AZ-900 (cloud concepts), AZ-104 (networking, NSGs), and AZ-204 (App Service, deployment, configuration)